

Kubernetes(K8S)-Class: 10

Helm in Kubernetes

What is Helm?

- Helm is the **package manager for Kubernetes**, just like **apt** for Ubuntu or **yum** for CentOS.
- It helps you **define, install, and manage Kubernetes applications**.
- Without Helm, you need to apply multiple YAML files manually (**kubectl apply -f ...**). Helm bundles them into **charts** for easier deployment.
- Helm also supports versioning and upgrades, making it easier to roll back if something goes wrong.
- **Helm can be installed** using official link: <https://helm.sh/docs/intro/install/>

Why is Helm needed in Kubernetes?

- **Simplifies deployments:** One command (**helm install**) can deploy an entire application stack (Deployment, Service, ConfigMaps, Ingress, etc.).
- **Parameterization:** Instead of editing YAML files manually, you use **values.yaml** to override configuration.
- **Reusability:** Teams can share charts via **Helm Repositories** (like DockerHub but for Helm charts).
- **Versioning:** Rollback to previous versions with **helm rollback**.
- **Standardization:** Developers get consistent deployments across environments (dev, staging, prod).

Helm Chart Structure

A **Helm chart** is a collection of files inside a directory. Example:

```
mychart/  
Chart.yaml    # Information about the chart  
values.yaml   # Default values for templates  
charts/       # Sub-charts (dependencies)  
templates/    # YAML templates for Kubernetes resources  
.helmignore   # Files to ignore when packaging chart
```

1. Chart.yaml (Metadata File)

- Defines **information about the chart**.
- Example:

OctoOps Academy

```
apiVersion: v2
name: my-app
description: A Helm chart for Kubernetes
type: application
version: 1.0
appVersion: "version:1"
```

Key Components:

- **apiVersion:** Always v2 for Helm 3.
 - **name:** Chart name.
 - **version:** Version of the chart itself.
 - **appVersion:** Version of the app being deployed (e.g., container image version).
-

2. values.yaml (Default Configurations)

- Stores **default configuration values** used in templates.
- Students can override them at runtime (**--set** or **-f custom-values.yaml**).
- Example:

```
app:
  dockerImage: octoopsacademy/octoops-app:v2
  containerName: ocotoops-container2
  deploymentName: ocotoops-app2

deploymentReplicas: 3

service:
  port: 3080
  nodePort: 31000
```

This file makes your chart **reusable** across environments without editing templates.

3. templates/ (Manifest Templates)

- Contains **Kubernetes resource definitions** (Deployment, Service, ConfigMap, etc.) but written as **templates**.
- Example: **templates/deployment.yaml**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{.Values.app.deploymentName}}
  labels:
    app: ocotoops
```

OctoOps Academy

```
spec:
  replicas: {{.Values.deploymentReplicas}}
  selector:
    matchLabels:
      app: ocotoops
  template:
    metadata:
      labels:
        app: ocotoops
    spec:
      containers:
        - name: {{.Values.app.containerName}}
          image: {{.Values.app.dockerImage}}
          ports:
            - containerPort: 80
```

Ex: Service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: ocotoops-nodeport
spec:
  type: NodePort
  selector:
    app: ocotoops
  ports:
    - port: 80
      targetPort: {{.Values.service.port}}
      protocol: TCP
      nodePort: {{.Values.service.nodePort}}
```

Note:

- {{.Values.* }} → Reads values from values.yaml.
- {{.Release.Name }} → Refers to Helm release name.
- {{.Chart.Name }} → Refers to chart name.

Common Helm Commands

Installing & Managing Charts

Install a chart: **helm install myapp ./mychart**

OctoOps Academy

Install with custom values: **helm install myapp ./mychart -f custom-values.yaml**

Generate a release name automatically: **helm install ./mychart --generate-name**

Checking Releases

List installed releases: **helm list**

Get status of a release: **helm status myapp**

list release rollouts/revisions: **helm history <release>**

See rendered manifests before installing: **helm template myapp ./mychart**

Upgrades & Rollbacks

Upgrade a release: **helm upgrade myapp ./mychart -f new-values.yaml**

Rollback to previous version: **helm rollback myapp 1**

Uninstall

Uninstall a release: **helm uninstall myapp**

Override values using command:

```
helm install myapp ./mychart --set deploymentReplicas =3, app.dockerImage =nginx
```

Kubernetes Dashboard & Metrics Server

What is Kubernetes Dashboard?

- **Kubernetes Dashboard** is a **web-based UI** to manage Kubernetes clusters.
- It allows you to:
 - See **pods, deployments, services, configmaps, secrets**.
 - Deploy new apps via UI.
 - Monitor cluster state.
 - View logs of pods.
 - If **Metrics Server** is installed → see **CPU / memory usage** of pods & nodes.

Without Dashboard, we mostly use **kubectl** CLI. Dashboard gives a **visual, user-friendly way** to explore and manage.

OctoOps Academy

What is Metrics Server?

- **Metrics Server** is a **lightweight aggregator** that collects **CPU & memory usage** data from all nodes and pods in the cluster.
- It pulls data from **Kubelets** running on each node.
- Required by:
 - **kubectl top nodes / kubectl top pods**
 - Horizontal Pod Autoscaler (HPA)
 - Kubernetes Dashboard (for graphs/metrics)
- Command to install metric server: **kubectl apply -f <https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml>**

Without Metrics Server → Dashboard works, but no resource usage stats.

Installing Kubernetes Dashboard

Step 1: Deploy Dashboard: Use below command (or follow instructions in official GitHub repo: <https://github.com/kubernetes/dashboard>)

```
kubectl apply -f
https://raw.githubusercontent.com/kubernetes/dashboard/v2.7.0/aio/deploy/recommended.yaml
```

This creates:

- **kubernetes-dashboard** namespace
 - Deployment, Service (ClusterIP), RBAC rules
-

Step 2: Create Service Account + ClusterRole + ClusterRoleBinding: To access the dashboard from the ui

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: dashboard
  namespace: default
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: clusterrole
rules:
- apiGroups: ["*"]
```

OctoOps Academy

```
resources: ["*"]
verbs: ["*"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: crb
subjects:
- kind: ServiceAccount
  name: dashboard
  namespace: default
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: clusterrole
```

Step 3: Get Token for Login

kubectl create token <sa-name> -n <ns_name>
ex: kubectl create token dashboard

Copy this token — required for login.
